极客时间

Skills 能力封装

把隐性经验变成可复用的标准化技能包

第 1 周 · 第 4 节

讲师：黄佳

极客时间黄佳老师出品

回顾：Sub-Agents 要点

极客时间

Sub-Agent = 独立角色 — 每个 Agent 有独立的 .md 文件定义身份

上下文隔离 — 采集 Agent 不知道分析 Agent 在做什么

最小权限 — 每个 Agent 只拥有完成任务所需的工具权限

我们的三个 Agent:

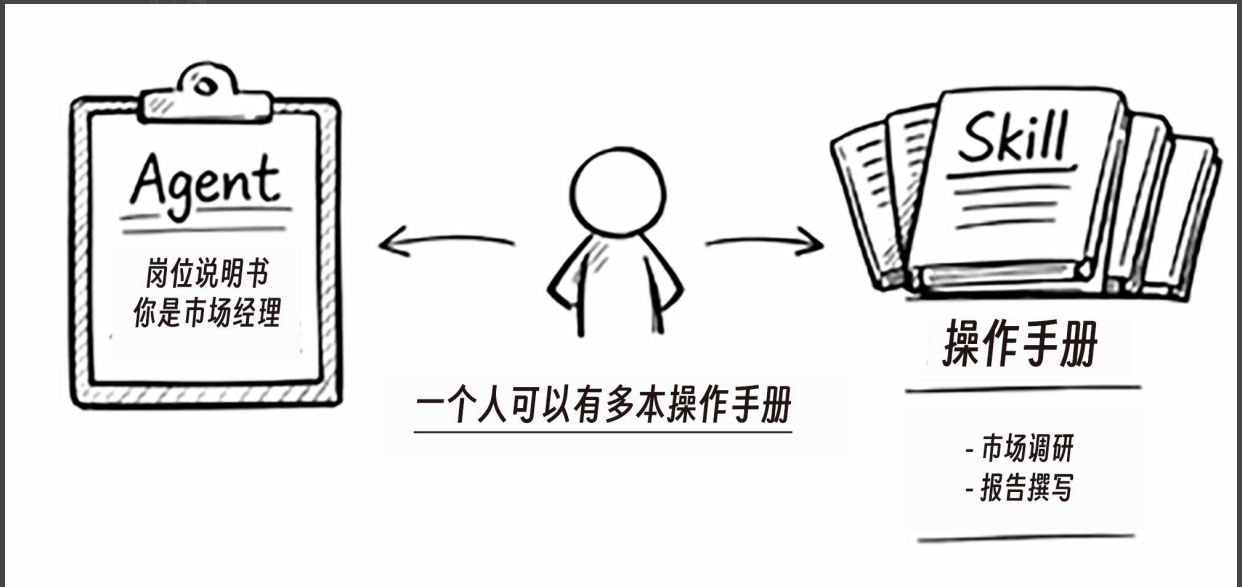
- collector.md — 采集 Agent，负责从外部源搜集原始数据
- analyzer.md — 分析 Agent，负责深度解读和评分
- organizer.md — 整理 Agent，负责去重、格式化和归档
-

遗留问题: Agent 知道“我是谁”，但不知道“怎么干”

- → 每次都要手动描述完整步骤 → 效率低、结果不一致

极客时间

Skill = 可复用的知识包



Agent 是角色（谁来干），Skill 是能力（怎么干）

Skill vs Agent 的区别

极客时间

Agent（角色）

定义：agents/*.md 文件
回答：「我是谁？」
内容：身份、职责、行为准则

示例：collector.md
你是采集 Agent
负责从外部源搜集数据
只使用 read、browser、write

类比：岗位说明书

Skill（能力）

定义：skills/*/SKILL.md 文件
回答：怎么干？
内容：步骤、标准、输入输出

示例：github-trending/SKILL.md
第 1 步：搜索热门仓库
第 2 步：提取仓库信息
第 3 步：过滤 →
第 ... 步 →
第 7 步：输出

类比：操作手册 / SOP

SKILL.md 格式详解

极客时间

```
---
name: github-trending                # 技能名称（简短唯一）
description: 当需要采集 GitHub 热门    # 语义触发条件
             开源项目时使用此技能
allowed-tools:                       # 权限白名单
  - read
  - browser
  - write
---

# GitHub 热门项目采集技能            <- 技能标题

## 使用场景                          <- 何时使用
在知识库采集阶段，从 GitHub 获取热门项目。

## 执行步骤                          <- 逐步操作（核心）
### 第 1 步：搜索热门仓库
### 第 2 步：提取仓库信息
### 第 3 步：过滤 ... 第 7 步：输出

## 注意事项                          <- 约束和限制
## 输出文件                          <- 结果存放路径
knowledge/raw/github-trending-YYYY-MM-DD.json
```

SKILL.md 各字段含义

极客时间

字段	位置	作用	示例
name	YAML	技能唯一标识	github-trending
description	YAML	语义触发条件（AI 据此匹配）	当需要采集 GitHub 热门项目时……
allowed-tools	YAML	权限白名单（可用工具列表）	read, browser, write
使用场景	正文	说明何时调用此 Skill	知识库采集阶段
执行步骤	正文	逐步操作指南（Skill 核心）	第 1 步：搜索 → 第 7 步：输出
注意事项	正文	约束、限制、常见错误	API 限频 10 次/分钟
输出文件	正文	结果存放路径和 JSON 格式	knowledge/raw/*.json

极客时间

深入理解 Skills

从本质到生态——Skills 不只是一个功能，而是一种范式

极客时间

极客时间

极客时间黄佳老师出品

极客时间

Skill 不是代码，是知识。

传统Tool/API

```
function doTask () {
  ...instructions..;
}
}
```

执行这段函数

按这个框架思考

Skill ≠ Tool — 本质区别

极客时间

Tool（工具调用）

本质：代码执行（函数调用）
执行者：客户端执行代码
接口：结构化 I/O（JSON Schema）
确定性：高（输入→输出固定）
类比：Claude 的「手」

例如：
read_file(path) → 文件内容
search_web(query) → 搜索结果

Skill（知识注入）

本质：知识注入（指令 + 上下文）
执行者：AI 自己推理执行
接口：自然语言（Markdown）
确定性：低（LLM 推理）
类比：Claude 的「教科书」

例如：
SKILL.md → 注入分析框架
AI 按框架思考 → 产出结果

Skills vs Sub-Agents vs context:fork — 三者关系

极客时间

Sub-Agent — 独立执行上下文，需要手动调度，知识在对话中（临时的）

Skill — 持久化知识包，AI 自动触发，知识在文件中（永久的）

Skill + context:fork — 两者合体：持久知识 + 自动创建临时子代理

怎么选？

- → 需要 AI 自己判断何时用 → Skill
- → 需要隔离执行环境 → Sub-Agent
- → 需要持久知识 + 隔离执行 → Skill + context:fork

类比：

- Sub-Agent = 公司内部专家（你要给他交代细节）
- Skill = 团队内部的 SOP 手册（写一次，所有人都能用）
- Skill + fork = SOP 手册 + 临时派人去独立执行

极客时间

极客时间黄佳老师出品

从 SOP 到组织智能 — Skill 的三级演进

极客时间

Level 1：SOP（可执行）

- 单个 SKILL.md + 固定步骤 → 完成一类标准任务
- 例：代码审查按 5 条固定规则检查

Level 2：专家系统（可扩展）

- SKILL.md + references/ + templates/ + scripts/
- 处理整个领域的 20+ 种变体
- 例：代码审查覆盖安全、性能、可维护性、API 规范

Level 3：组织智能（可规模化）

- 多个 SubAgent + 各自专属 Skills + 流水线协作
- 例：分析 Agent → 审核 Agent → 测试 Agent → 部署 Agent

极客时间

声明式的生命力 — 为什么 Skills 会长期存在？

- **1956 正则表达式** — 声明匹配模式 → 存活 70 年
- **1974 SQL** — 声明要什么数据 → 存活 50 年
- **2025 SKILL.md** — 声明要做什么 → 会存活多久？

长寿声明式技术的三个共性：

1. 与执行引擎解耦 — SQL 在 MySQL/PG/SQLite 都能跑
   → SKILL.md 在 Claude/GPT/Gemini 都能用（27+ 平台支持）
2. 直接映射人类意图 — SQL 像说人话，不像写机器码
   → SKILL.md 就是 Markdown，非工程师也能写
3. 引擎升级不影响声明 — SQL 不变，优化器越来越快
   → SKILL.md 不变，模型从 GPT-3.5 升到 Claude 4 自动变好

极客时间黄佳老师出品

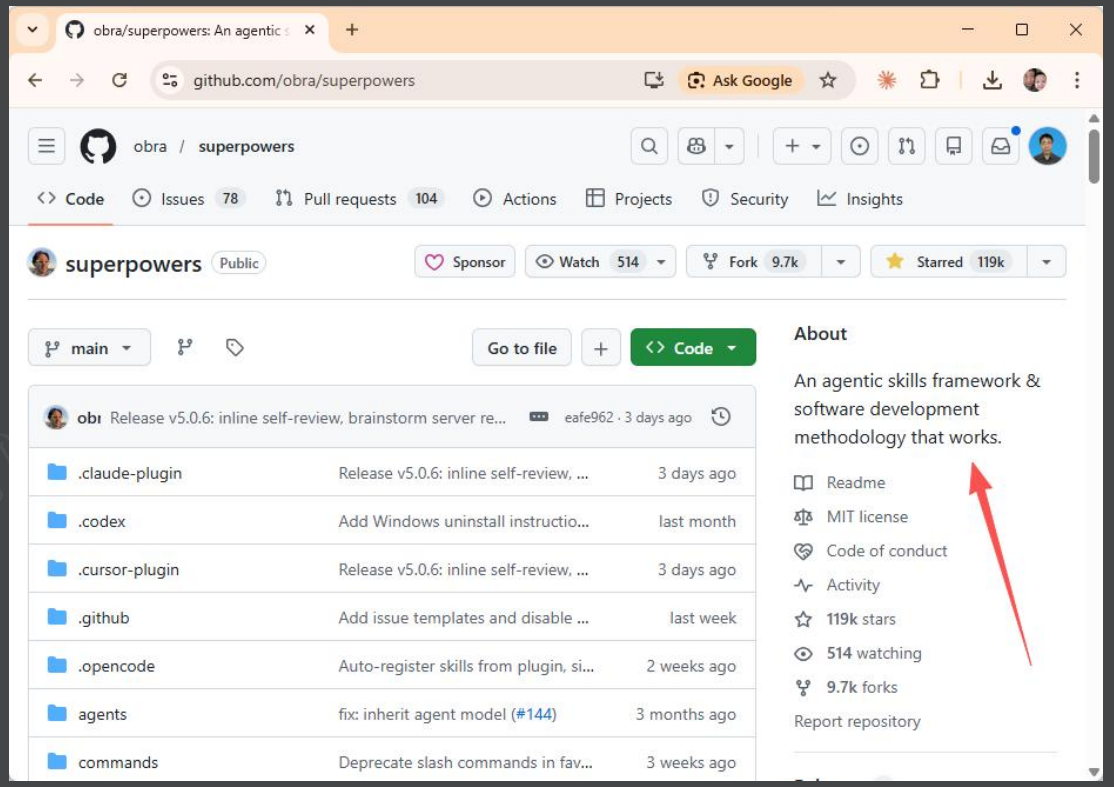
Superpowers — 官方标准技能包

Superpowers 官方标准技能包（适用于 Claude Code 等工具）：

- superpowers:brainstorm — 头脑风暴，生成 brainstorm.md
- superpowers:plan — 执行规划，生成 plan.md
- superpowers:specs — 技术规格，生成 specs.md
- superpowers:code — 按规格编码实现
- superpowers:debugging — 系统化调试
- superpowers:TDD — 测试驱动开发
- superpowers:code-review — 代码审查

与自定义 Skill 的关系：

- 命名空间隔离 — superpowers:* vs 你的 Skill（无前缀）
- 互不冲突 — AI 按语义匹配最相关的那个
- 企业可在 superpowers 上叠加公司规范层（不修改原插件）



极客时间黄佳老师出品

Skills 生态 — 27+ 平台支持的行业标准

极客时间

SKILL.md 已是事实上的行业标准格式:

- Claude Code · OpenAI Codex CLI · Google Gemini CLI
- Cursor · GitHub Copilot · VS Code · Windsurf
- Manus · Goose · Letta · Trae · 通义灵码 ...

生态工具:

- Vercel skills.sh — 类似 npm 的 Skill 包管理器
- anthropics/claude-plugins-official — 官方插件仓库
- 社区贡献 — 各种领域的开源 Skill 包

意味着什么?

- 你今天写的 SKILL.md 不是锁定在某一个工具里的
- 换工具、换模型，Skill 直接复用
- → 你投入在 Skill 上的时间不会浪费

极客时间黄佳老师出品

企业视角：Skills = 可执行的组织知识资产

极客时间

传统企业知识管理的困境：

- Confluence 上的 Wiki → 没人看 → 知识腐烂
- 老员工离职 → 隐性经验流失 → 新人重复踩坑

Skills 解决了什么？

- 把隐性经验（老员工脑子里的东西）→ 变成显性 SKILL.md
- 不只是文档，而是可被 AI 执行的操作手册
- 新人不需要找老员工问 → AI 自动按 Skill 执行

三层知识分发：

- ~/.claude/skills/ → 个人级（你自己的习惯）
- .claude/skills/ 或者 .opencode/skills/ → 项目级（团队共享）
- Plugin 分发 → 企业级（全组织标准化）
- 优先级：企业 Plugin > 个人 > 项目

极客时间黄佳老师出品

高频问答 — 学员最常问的 3 个问题

极客时间

Q: Skill 太多会不会超出上下文?

- 一般不会。渐进式加载只扫 description（~5 tokens/Skill），
- 总预算 15,000 字符。超过的 Skill description 会被截断。
- 实战建议：控制在 30 个以内，超过就该用脚本替代了。

Q: 自动触发不靠谱，总是触发错的 Skill?

- 现阶段确实如此。实战经验：用 /command 显式调用更稳。
- 把 user-invocable: true 打开，先跑稳再考虑自动触发。
- 自动触发是未来方向，但现在显式调用是最佳实践。

Q: Skill 和 MCP 都能扩展能力，怎么选?

- Skill 注入知识（教 AI 怎么想），MCP 注入工具（给 AI 新手）。
- 先用 Skill。只有当 AI 需要访问外部系统时才加 MCP。
- 口诀：知识用 Skill，工具用 MCP，两者正交互补。

极客时间黄佳老师出品

极客时间

实战：封装 2 个 Skill

github-trending（采集） + tech-summary（分析）

极客时间黄佳老师出品

Skill 1：github-trending（采集 GitHub Trending 数据）

极客时间

目的 — 从 GitHub 搜索并采集 AI 领域热门开源项目

调用者 — 采集 Agent（collector.md）

权限 — read、browser、write

7 步流程：

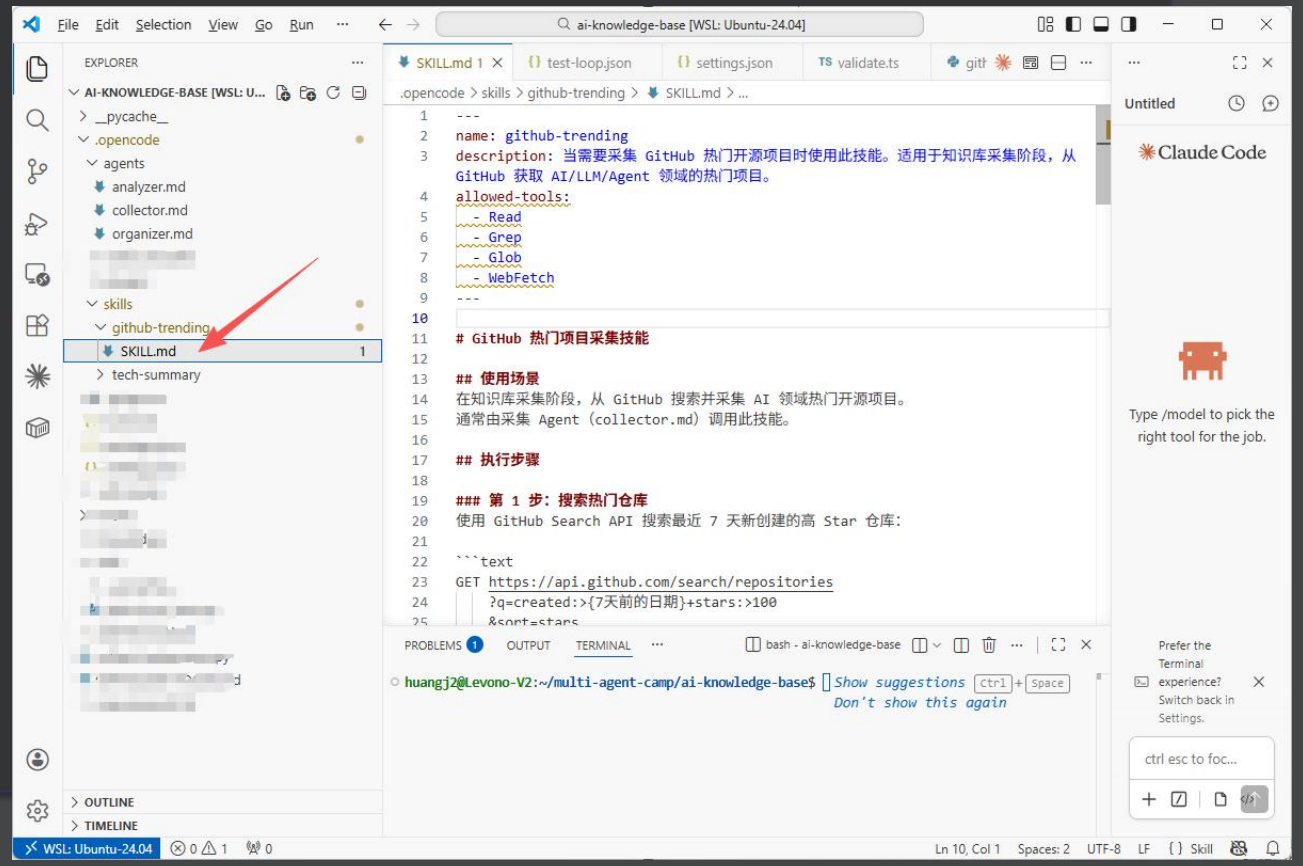
- 第 1 步：搜索热门仓库（GitHub Search API + 筛选参数）
- 第 2 步：提取仓库信息（名称、Star、描述、语言、话题）
- 第 3 步：过滤（纳入 AI/LLM/Agent；排除 Awesome 列表/纯教程）
- 第 4 步：去重（多次搜索的重复结果只保留一条）
- 第 5 步：撰写一句话中文摘要（公式：做什么 + 为什么值得关注）
- 第 6 步：排序取 Top 15（按 Star 数降序）
- 第 7 步：输出 JSON 到 knowledge/raw/github-trending-YYYY-MM-DD.json

极客时间黄佳老师出品

github-trending/SKILL.md 核心内容

skills/github-trending/SKILL.md | SKILL.md

```
---
name: github-trending
description: 当需要采集 GitHub 热门开源项目时使用此技能
allowed-tools: [read, browser, write]
---
# GitHub 热门项目采集技能

### 第 1 步：搜索热门仓库
GET https://api.github.com/search/repositories
  ?q=created:>YYYY-MM-DD+stars:>100
  &sort=stars&order=desc&per_page=30
也可按主题搜索：topic:ai / topic:llm / topic:agent

### 第 3 步：过滤
纳入：AI/ML/LLM/Agent、开发者工具、框架重大更新
排除：Awesome 列表、纯教程、Star 刷量、无 README

### 第 5 步：撰写一句话中文摘要
公式：[项目名] 做什么 + 为什么值得关注
示例：OpenClaw 是开源 Agent 运行时，支持 50+ 平台路由编排

### 第 7 步：输出结果
→ knowledge/raw/github-trending-YYYY-MM-DD.json
{"source":"github","total_after_filter":15,
 "items":[{name, url, stars, summary, ...}]}
```



Skill 2：tech-summary（技术摘要生成）

极客时间

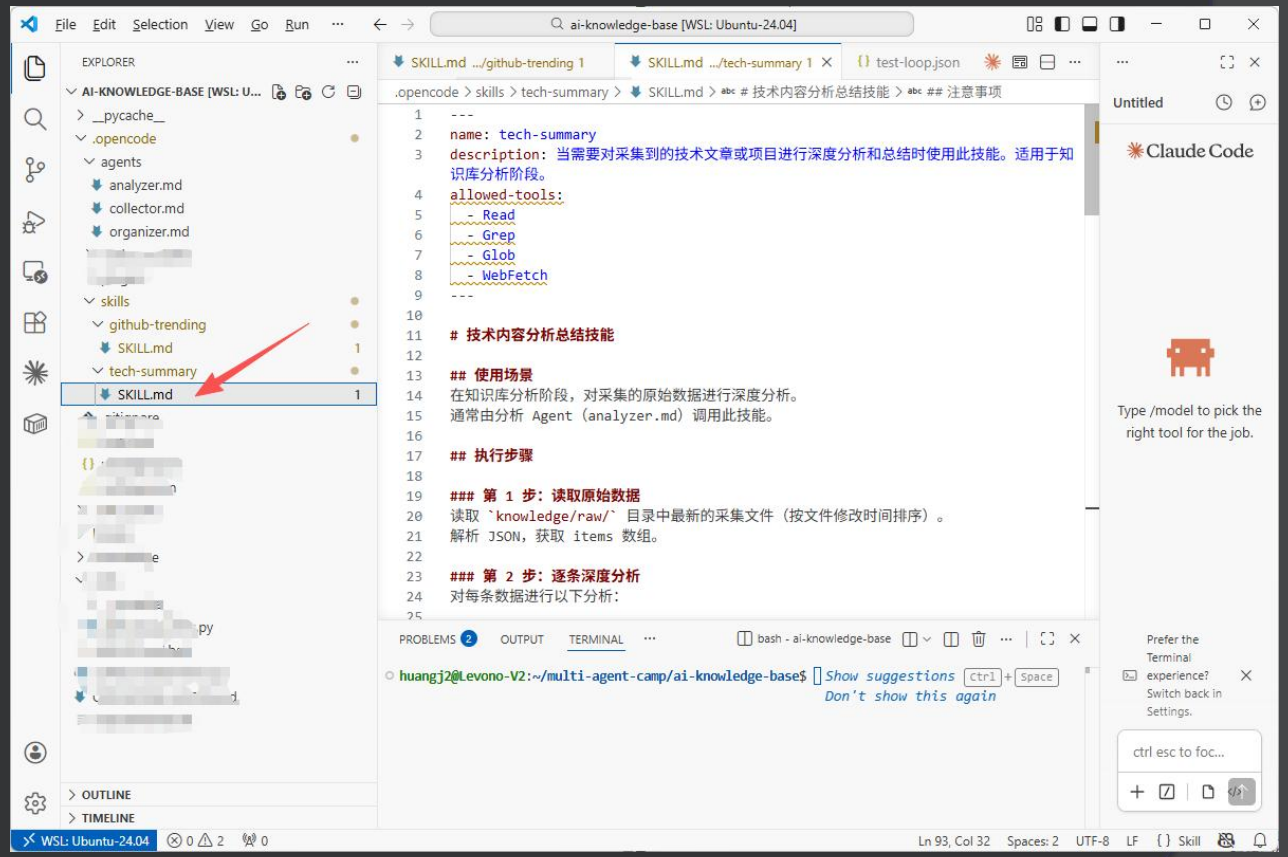
- **目的** — 对采集到的原始数据做深度分析，产出结构化知识条目
- **调用者** — 分析 Agent（analyzer.md）
- **权限** — read、browser、grep、glob
- **4 步流程：**
- 第 1 步：读取原始数据（knowledge/raw/ 最新采集文件）
- 第 2 步：逐条深度分析
  - 2.1 一句话摘要（≤50 字，有具体信息）
  - 2.2 技术亮点提取（2–3 个核心点，用事实说话）
  - 2.3 架构分析（如适用）
  - 2.4 相关性评分（1–10，要有区分度）
  - 2.5 标签建议（从预定义列表选 2–5 个）
- 第 3 步：趋势发现（本批数据的共同主题和方向）
- 第 4 步：输出分析结果 JSON

极客时间

tech-summary/SKILL.md 核心内容

skills/tech-summary/SKILL.md | SKILL.md

```
---
name: tech-summary
description: 当需要对技术文章或项目进行深度分析时使用
allowed-tools: [read, browser, grep, glob]
---
# 技术内容分析总结技能

### 第 1 步：读取原始数据
读取 knowledge/raw/ 中最新的采集文件

### 第 2 步：逐条深度分析
2.1 一句话摘要（≤50 字，有具体信息）
    公式：[名称] 是 [做什么]，[为什么值得关注]
2.2 技术亮点（2-3 个核心点）
    用事实："推理速度提升 3 倍" 不是 "速度很快"
2.3 相关性评分（1-10）
    9-10 改变行业格局 | 7-8 直接有帮助
    5-6  值得了解     | 1-4 可略过

### 第 3 步：趋势发现
本批有几个项目涉及同一方向？出现了什么新概念？

### 第 4 步：输出分析结果
{"items":[{source_id, summary, tech_highlights,
  relevance_score, suggested_tags}], "trends":[...]}
```



老师出品

语义触发机制 — description 字段如何决定 Skill 激活

极客时间

原理： AI 扫描所有 SKILL.md 的 description，做语义匹配

触发示例：

- 你说："帮我看看 GitHub 最近有什么热门 AI 项目"
- AI 匹配到 → github-trending（描述含'采集 GitHub 热门开源项目'）

- 你说："对采集数据做深度分析"
- AI 匹配到 → tech-summary（描述含'深度分析总结'）

description 写作要点：

- 包含关键动作词 — 采集/分析/整理/生成
- 包含关键对象 — GitHub/HN/技术文章/知识条目
- 足够具体以区分不同 Skill — 避免两个 Skill 描述太相似

渐进式加载： 先加载目录（每个 Skill ~5 tokens），需要时才加载全文

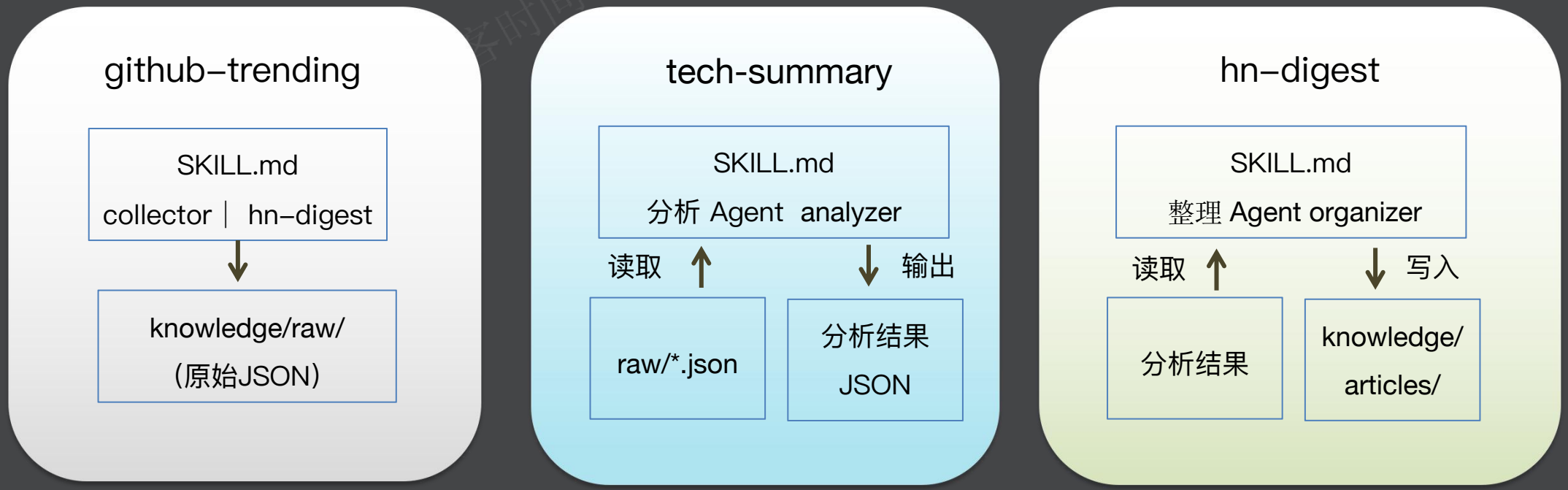
极客时间黄佳老师出品

V1 知识库跑通

编码自动化 采集 → 分析 → 整理，产出第一批结构化知识

极客时间黄佳老师出品

V1 完整流程：编码自动化 采集 → 分析 → 整理



--------采集 ----------分析----------整理----------

演示：用 OpenCode 触发 Skill，产出第一批知识

极客时间

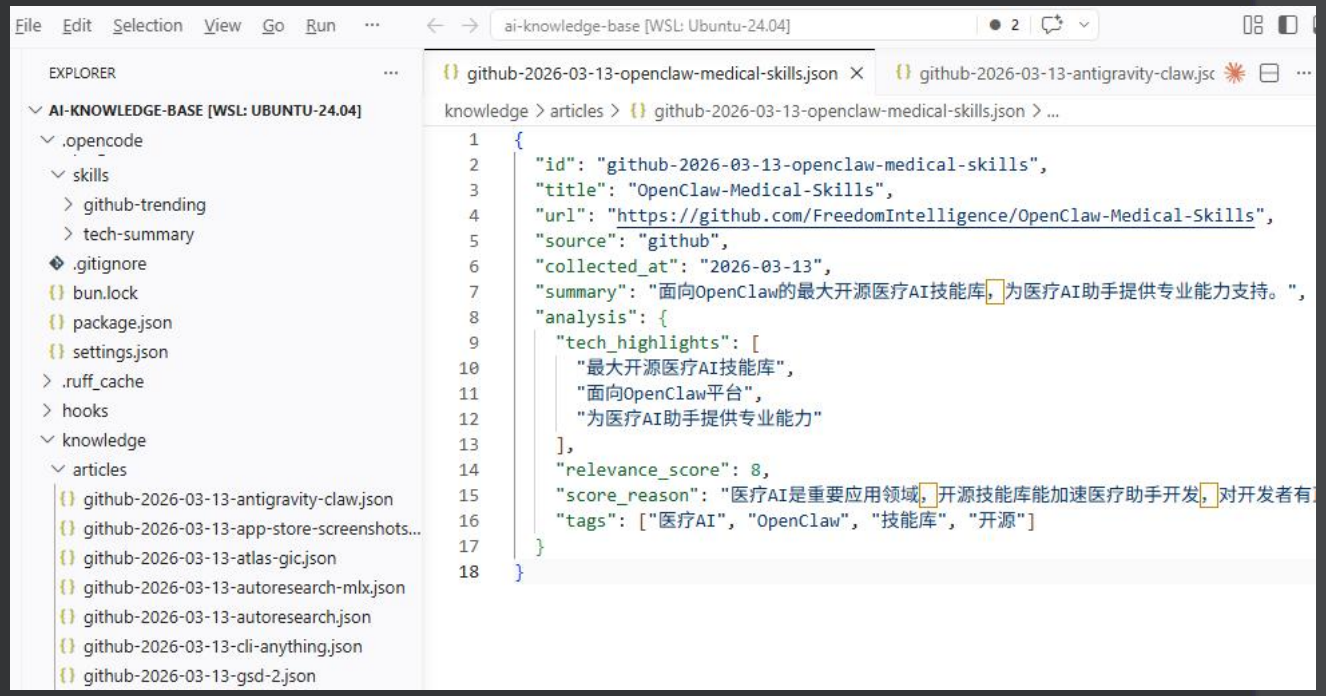
project/ai-knowledge-base/ | OpenCode 对话示例

```
# ── Step 1：采集 ──────────────────────────────────────────────
你: "用采集 Agent 的角色（agents/collector.md），
     调用 github-trending 技能，
     采集本周 AI 领域的热门项目"

# → OpenCode 加载 collector.md + github-trending SKILL.md
# → 按 7 步执行，输出 knowledge/raw/github-trending-2026-03-09.json

# ── Step 2：分析 ──────────────────────────────────────────────
你: "用分析 Agent（agents/analyzer.md），
     对 knowledge/raw/ 中最新的采集数据做深度分析"

# → 输出结构化分析结果 JSON（摘要 + 亮点 + 评分 + 趋势）

# ── Step 3：整理 ──────────────────────────────────────────────
你: "用整理 Agent（agents/organizer.md），
     将分析结果整理归档到 knowledge/articles/"

# → 去重 + 格式化 + 分类归档
# 三步完成：原始数据 → 深度分析 → 结构化知识
```

V1 产出展示：JSON 知识条目示例

knowledge/raw/github-trending-2026-03-09.json | json


```
{
  "source": "github",
  "collected_at": "2026-03-09T10:30:00Z",
  "total_fetched": 30,
  "total_after_filter": 15,
  "items": [
    {
      "name": "opencode",
      "full_name": "nicepkg/opencode",
      "url": "https://github.com/nicepkg/opencode",
      "description": "A powerful CLI AI coding tool",
      "summary": "OpenCode 是开源 AI 编程终端工具，
              支持 100+ 模型的本地化 Agent 开发",
      "stars": 102000,
      "language": "TypeScript",
      "topics": ["ai", "cli", "coding-agent"],
      "relevance_score": 9,
      "tech_highlights": [
        "MIT 开源，支持所有主流模型",
        "完全兼容 Claude Code 的扩展机制"
      ]
    }
  ]
}
```

本节作业

极客时间

任务 1：封装一个自定义 Skill

参考 github-trending 和 tech-summary 的格式，自己设计一个新的 SKILL.md。可选方向：arxiv-papers（采集 arXiv 最新 AI 论文）、producthunt-daily（Product Hunt 日榜）、weixin-tech（微信公众号技术文章采集）

要求：YAML 元数据完整、步骤原子化、输入输出明确。

任务 2：跑通 V1 完整流程（采集 + 分析）

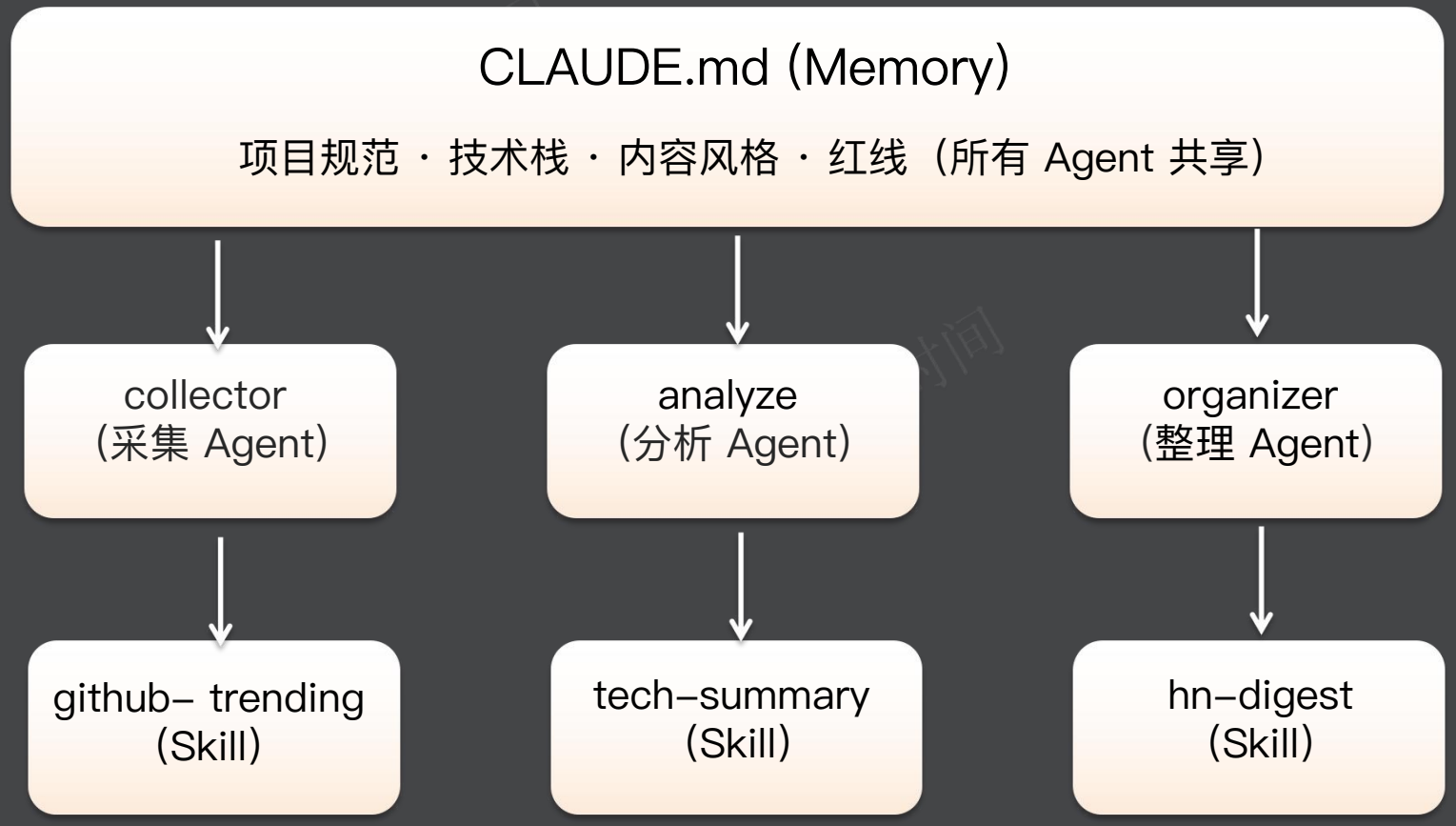
在 OpenCode 中分别触发 github-trending 和 tech-summary 技能，完成一次完整的“采集 → 分析”流程。检查产出 JSON 的格式是否符合 SKILL.md 定义：字段全不全？摘要符不符合公式？评分有没有区分度？不满意就修改 SKILL.md 后重新执行——这就是 Skill 工程的迭代循环。

任务 3：（可选挑战）对比有 Skill 和无 Skill 的执行效果

用同样的需求分别测试两种方式：1）不加载 Skill，纯自然语言描述需求；2）加载 Skill 后执行。对比产出的 JSON 格式一致性、内容质量、执行时间。把对比结果写成简短总结——这就是 Skill 的价值证明。

极客时间黄佳老师出品

V1 项目全景回顾（Memory + Sub-Agents + Skills）



极客时间黄佳老师出品

第 1 周总结：你刚刚建了什么？

极客时间

极客时间

SDD 层级	你写的文件	回答的问题	课节
项目规范	AGENTS.md	项目是什么？怎么做？	第 2 节
角色规范	agents/*.md（3个）	谁来干？权限边界？	第 3 节
能力规范	skills/*/SKILL.md（2个）	具体怎么干？步骤？	第 4 节

V1 产出清单

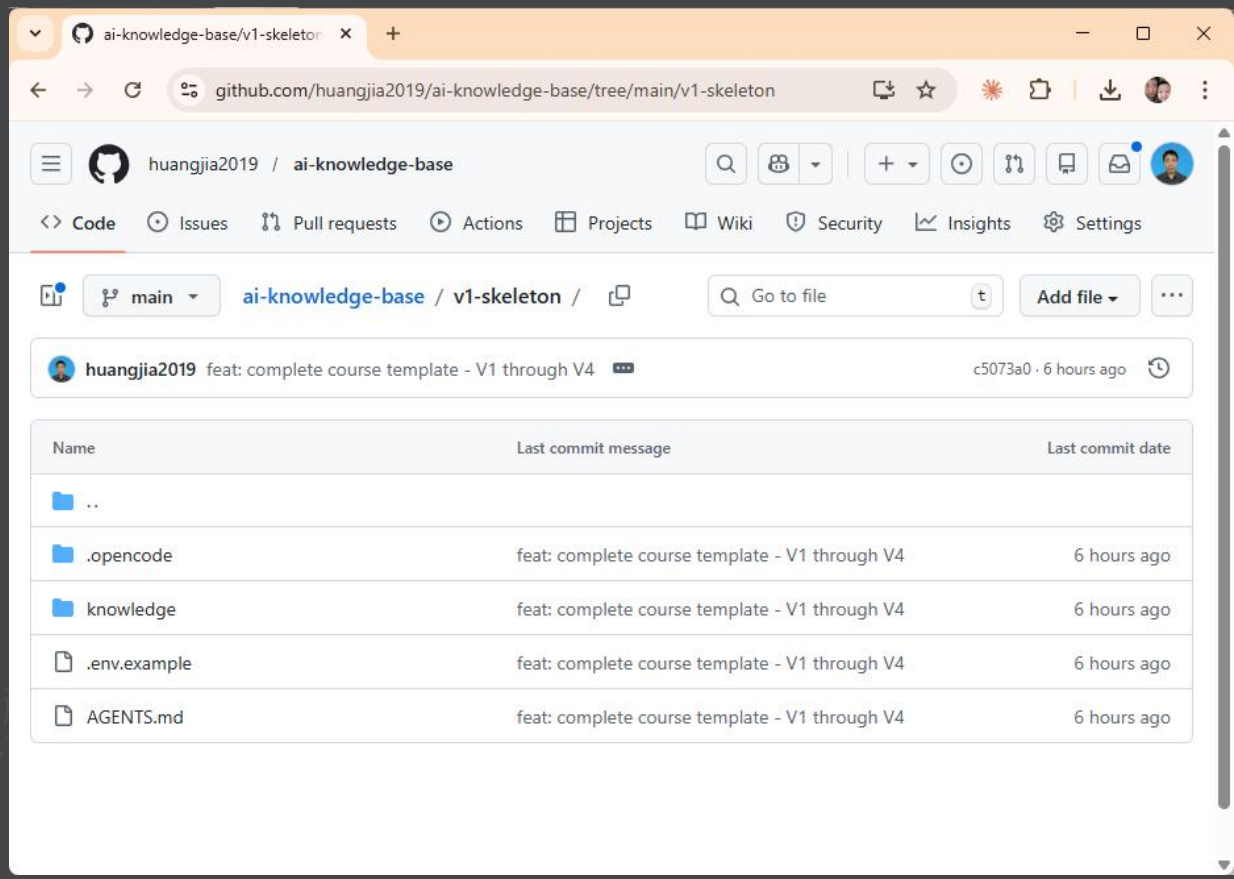
SDD 三层规范：

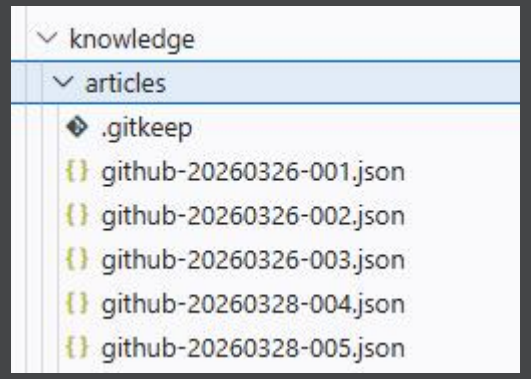
- AGENTS.md — 项目规范（SDD 第一层）
- .opencode/agents/ — 3 个角色定义文件（SDD 第二层）
- .opencode/skills/ — 2 个技能文件（SDD 第三层）

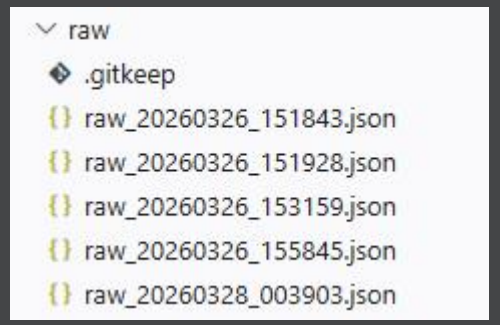
数据产出：

- knowledge/raw/ — 第一批采集数据
- knowledge/articles/ — 第一批结构化知识条目

下周的跃迁：从“对话驱动”到“自动运转”







极客时间黄佳老师出品

本节小结

极客时间

Skill = 声明式能力封装（SKILL.md）

一个 Skill 解决一类问题，可跨项目复用

三步走：采集 → 分析 → 整理，Skill 定义好一切

V1 编码自动化：用 AI 搭建骨架，人驱动对话执行

下一节：第 2 周 · 自动化工程 — Hooks + Pipeline + CI/CD